TASK 1 — UNDERSTAND & VISUALIZE

Transformer Architecture and Pretrained Sentiment Analysis

Student / Project Name	Dharshini.M · Task 1
Repository Name	dharshini-m-task1
Selected Architecture	Transformer (DistilBERT)
Inference Task	English sentiment classification

Objective

To build a conceptual understanding of the Transformer architecture and demonstrate basic hands-on inference using a pretrained model. The experiment uses a pretrained sentiment-analysis model to classify five sample sentences as positive or negative.

1. Transformer Architecture — Explanation (approximately 330 words)

A Transformer is a deep-learning architecture designed to process sequences such as sentences. It is widely used in natural-language processing because it can represent relationships between words and use context to interpret meaning. Unlike a traditional recurrent network, a Transformer can process tokens in parallel during training, making it effective for many language tasks.

The first step is tokenization. The input sentence is divided into tokens, which may be complete words or smaller word pieces. Each token is mapped to a numerical token ID. Since a neural network works with numbers, the IDs are passed to an embedding layer. The embedding layer represents each token as a vector. Positional information is also included so that the model can use the order of tokens in the sentence.

A central component of the Transformer is self-attention. It allows the representation of one token to use information from other tokens in the same input. This is important because a word's meaning can depend on its context. For example, the word "not" changes the meaning of "good" in the sentence "The movie was not good." Attention helps the model represent these relationships instead of considering every word independently.

Transformer encoder blocks commonly contain an attention sublayer and a feed-forward neural network. Residual connections and normalization support stable information flow through the network. In a pretrained sentiment classifier, the encoder creates contextual representations of the input. A classification head then uses these representations to produce scores for sentiment classes such as POSITIVE and NEGATIVE. The class with the highest score is returned as the prediction.

In this assignment, the pretrained DistilBERT model is used for inference. The model has already learned useful language patterns during its prior training and fine-tuning. Five new sentences are passed through its tokenizer and model, and the resulting sentiment labels are reported. This experiment demonstrates how a pretrained architecture can be applied to a practical language task without training a new model. The scores can help indicate how strongly the model favors its chosen class, but they do not guarantee that a prediction is correct.

2. Labelled Architecture Diagram

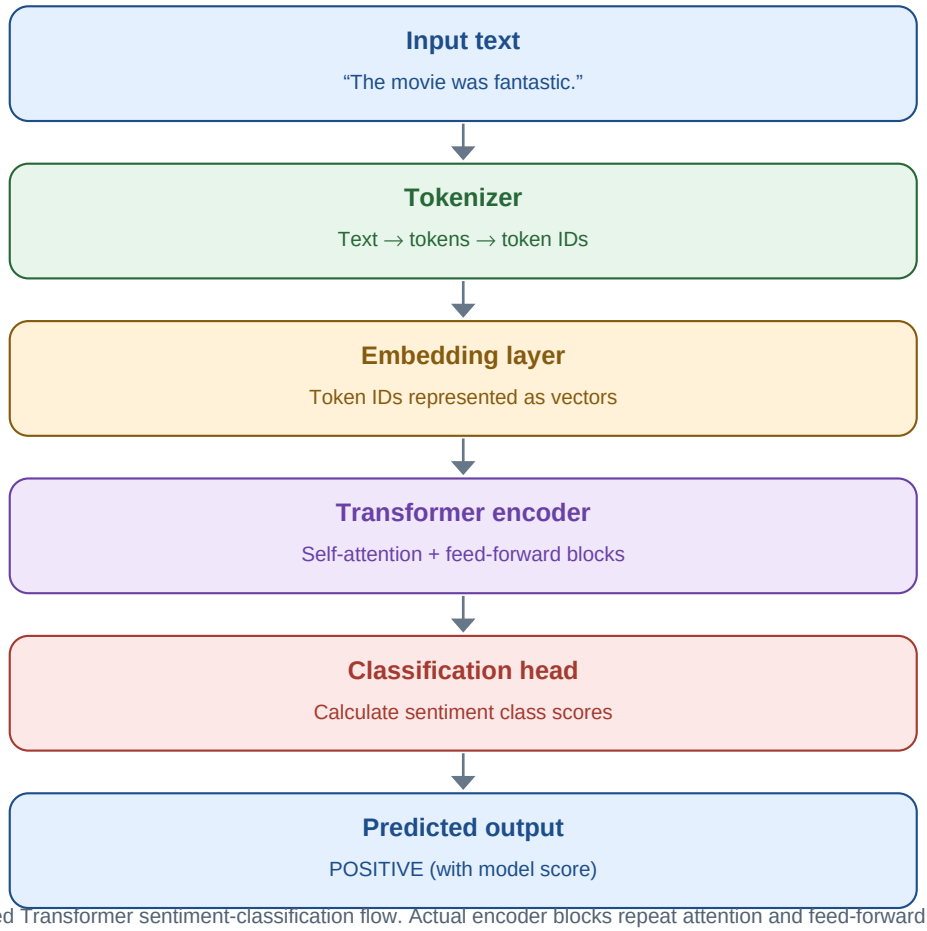

Figure 1. Simplified Transformer sentiment-classification flow. Actual encoder blocks repeat attention and feed-forward sublayers and include residual connections and normalization.

3. Pretrained Model and Implementation

Model: distilbert/distilbert-base-uncased-finetuned-sst-2-english
Library: Hugging Face Transformers with PyTorch
Task: Sentiment analysis (POSITIVE / NEGATIVE)

Create a Python file named **inference.py** and run it in an environment with the required packages installed. The first run may download model files and needs internet access.

```
# Install dependencies in your terminal:
# pip install transformers torch

from transformers import pipeline

classifier = pipeline(
    "sentiment-analysis",
    model="distilbert/distilbert-base-uncased-finetuned-sst-2-english"
)

sample_inputs = [
    "I really enjoyed this movie.",
    "The food was terrible and tasteless.",
    "The weather is beautiful today.",
    "I am disappointed with the service.",
    "The book was interesting and inspiring."
]

results = classifier(sample_inputs)

for i, (sentence, result) in enumerate(
    zip(sample_inputs, results), start=1
):
```

```
    print(f"Input {i}: {sentence}")
    print(f"Predicted Sentiment: {result['label']}")
    print(f"Confidence Score: {result['score']:.4f}")
    print("-" * 50)
```

Run with: **python inference.py**

4. Inference Results

The model is expected to identify the broad sentiment of the following sample inputs. The labels below are the intended result pattern; the Python script prints the actual model labels and confidence scores during execution. Record those values if your instructor requires the exact run output.

#	Input sentence	Expected label
1	I really enjoyed this movie.	POSITIVE
2	The food was terrible and tasteless.	NEGATIVE
3	The weather is beautiful today.	POSITIVE
4	I am disappointed with the service.	NEGATIVE
5	The book was interesting and inspiring.	POSITIVE

Result Discussion

The five sentences include both positive and negative opinions. The expected pattern is three positive predictions and two negative predictions. This test illustrates that a pretrained Transformer can use the wording and context of a sentence to assign a sentiment label. Confidence scores vary by input and should be interpreted as model scores rather than certainty. If any actual prediction differs from the expected label, report that difference honestly and briefly consider whether the wording is ambiguous or the model may have misclassified it.

5. Conclusion

This assignment introduced the Transformer architecture and its main components: tokenization, embeddings, self-attention, encoder processing, and classification. A pretrained DistilBERT model was selected to perform sentiment analysis on five sample sentences. The inference experiment demonstrates how a trained model can be reused to produce predictions on new text. It also shows the importance of checking model outputs rather than assuming every prediction is correct.

6. Repository and Submission

Use the repository name **dharshini-m-task1** and include these files:

```
dharshini-m-task1/
■■■ README.md
■■■ inference.py
■■■ requirements.txt
```

Suggested requirements.txt:

```
transformers
torch
```

Upload the files to GitHub, verify that the README and script are visible, then submit the repository URL in the Task 1 submission field in the Student LMS.

Note: This PDF contains the written assignment and code. Run the script to obtain the machine-generated confidence scores for your own execution; scores are intentionally not fabricated in this report.